Application to linear systems: Kalman filter

- We now apply general Gaussian sequential probabilistic inference solution to specific case of a linear system.
- Linear systems have desirable property that all pdfs remain Gaussian if stochastic inputs are Gaussian, so the assumptions made in deriving filter steps hold exactly.
- The linear Kalman filter assumes that the system being modeled can be represented in the discrete-time "state-space" form:

$$x_k = A_{k-1}x_{k-1} + B_{k-1}u_{k-1} + w_{k-1}$$
$$z_k = C_k x_k + D_k u_k + v_k.$$

- Input is u_k, output is z_k, state is x_k; process noise is w_k, sensor noise is v_k.

Assumptions on noises

- We assume that w_k and v_k are mutually uncorrelated white Gaussian random processes, with zero mean and covariance matrices having known value:

$$\mathbb{E}[w_n w_k^T] = \begin{cases} \Sigma_{\tilde{w}}, & n = k; \\ 0, & n \neq k. \end{cases} \qquad \mathbb{E}[v_n v_k^T] = \begin{cases} \Sigma_{\tilde{v}}, & n = k; \\ 0, & n \neq k, \end{cases}$$

  and $\mathbb{E}[w_k x_0^T] = 0$ for all k.
- The assumptions on the noise processes w_k and v_k and on the linearity of system dynamics are rarely (never) met in practice, but the consensus of the literature and practice is that the method still works very well.

Deriving the linear Kalman filter (1a)

- **KF step 1a**: State prediction time update.
  - Here, we compute the predicted state:

$$\begin{aligned} \hat{x}_k^- &= \mathbb{E}\big[f(x_{k-1}, u_{k-1}, w_{k-1}) \mid \mathbb{Z}_{k-1}\big] \\ &= \mathbb{E}\big[A_{k-1}x_{k-1} + B_{k-1}u_{k-1} + w_{k-1} \mid \mathbb{Z}_{k-1}\big] \\ &= \mathbb{E}\big[A_{k-1}x_{k-1} \mid \mathbb{Z}_{k-1}\big] + \mathbb{E}\big[B_{k-1}u_{k-1} \mid \mathbb{Z}_{k-1}\big] + \mathbb{E}\big[w_{k-1} \mid \mathbb{Z}_{k-1}\big] \\ &= A_{k-1}\hat{x}_{k-1}^+ + B_{k-1}u_{k-1}, \end{aligned}$$

    by the linearity of expectation, noting that w_{k-1} is zero-mean.
  - This is no longer an abstract mathematical concept/equation. We can implement this in code!
- **INTUITION**: When predicting the present state given only past measurements, the best we can do is to use the most recent state estimate and system model, propagating the mean forward in time.

Deriving the linear Kalman filter (1b)

- **KF step 1b**: Prediction-error covariance time update.
  - First, we note that the prediction error is $\tilde{x}_k^- = x_k - \hat{x}_k^-$, so,

$$\begin{aligned}\tilde{x}_k^- &= x_k - \hat{x}_k^- = A_{k-1}x_{k-1} + B_{k-1}u_{k-1} + w_{k-1} - A_{k-1}\hat{x}_{k-1}^+ - B_{k-1}u_{k-1} \\ &= A_{k-1}\tilde{x}_{k-1}^+ + w_{k-1}.\end{aligned}$$

  - Therefore, the covariance of the prediction error is:

$$\begin{aligned}\Sigma_{\tilde{x}_k}^- &= \mathbb{E}\big[(\tilde{x}_k^-)(\tilde{x}_k^-)^T\big] = \mathbb{E}\big[(A_{k-1}\tilde{x}_{k-1}^+ + w_{k-1})(A_{k-1}\tilde{x}_{k-1}^+ + w_{k-1})^T\big] \\ &= \mathbb{E}\big[A_{k-1}\tilde{x}_{k-1}^+(\tilde{x}_{k-1}^+)^T A_{k-1}^T + w_{k-1}(\tilde{x}_{k-1}^+)^T A_{k-1}^T + A_{k-1}\tilde{x}_{k-1}^+ w_{k-1}^T + w_{k-1}w_{k-1}^T\big] \\ &= A_{k-1}\Sigma_{\tilde{x},k-1}^+ A_{k-1}^T + \Sigma_{\widetilde{w}}.\end{aligned}$$

  - The cross terms drop out of the final result since the white process noise w_{k-1} is not correlated with the state at time $k-1$ and is zero mean.
- Again, this is something we can implement in code! (which will be a recurring theme.)

Intuition for step 1b

- **INTUITION:** When estimating state-prediction error covariance:

$$\Sigma_{\tilde{x}_k}^- = A_{k-1}\Sigma_{\tilde{x},k-1}^+ A_{k-1}^T + \Sigma_{\widetilde{w}}.$$

  - Best to use most recent covariance estimate and propagate forward in time.
  - For stable systems, $A_{k-1}\Sigma_{\tilde{x},k-1}^+ A_{k-1}^T$ is contractive: covariance gets "smaller."
    - State of stable systems always decays toward zero in absence of input, or toward a deterministic trajectory if $u_k \neq 0$.
    - Term tells us that we tend to get increasingly certain of state estimate over time.
  - On the other hand, $\Sigma_{\widetilde{w}}$ adds to the covariance.
    - Unmeasured inputs add uncertainty to our estimate because they perturb the trajectory away from the known trajectory based only on u_k.

Deriving the linear Kalman filter (1c)

- **KF step 1c**: Predict system output.
  - We predict the system output as:

$$\begin{aligned}\hat{z}_k &= \mathbb{E}\big[h(x_k, u_k, v_k) \mid \mathbb{Z}_{k-1}\big] = \mathbb{E}\big[C_k x_k + D_k u_k + v_k \mid \mathbb{Z}_{k-1}\big] \\ &= \mathbb{E}\big[C_k x_k \mid \mathbb{Z}_{k-1}\big] + \mathbb{E}\big[D_k u_k \mid \mathbb{Z}_{k-1}\big] + \mathbb{E}\big[v_k \mid \mathbb{Z}_{k-1}\big] \\ &= C_k\hat{x}_k^- + D_k u_k,\end{aligned}$$

    since v_k is zero-mean.
  - By now it is redundant to mention, but to be as clear as possible we notice again that this is something we can implement in code!
- **INTUITION**: $\hat{z}_k$ is best guess of system output, given only past measurements.
  - The best we can do is to predict the output given the output equation of the system model, and our best guess of the system state at the present time.

Summary

- We have started to specialize the general Gaussian sequential-probabilistic-inference steps to the special case of a linear system to develop the Kalman-filter equations.
- The general steps are not implementable as computer programs, since they involve statistical operations (expected values).
- However, the specialized Kalman-filter steps we have developed so far are completely valid program operations.
- By examining the equations, we can gain insight into the operation of a Kalman filter and have confidence that it is performing reasonable actions.